INACAP
Sede Temuco
Área Informática y Telecomunicaciones
Analista Programador

Gestión de Proyectos tipo Trello

Diagrama de clases UML y justificación del diseño

Proyecto 3 · Programación Orientada a Objetos

Integrantes: Halbert Miranda
Benjamin Arriagada

Asignatura: Programación Orientada a Objetos
Docente: Erick Bailey

Fecha: 19 de octubre de 2026 · Versión 5

Índice

1. Introducción 3

2. Contexto y alcance 3

3. Identificación de clases 4

4. Diagrama de clases 5

5. Relaciones y multiplicidades 6

6. Decisiones de diseño principales 6

6.1 UsuarioProyecto como clase de asociación 6

6.2 El responsable de una tarjeta es un participante, no un usuario cualquiera 7

6.3 Cadena de composición Proyecto → Tablero → Lista → Tarjeta 7

6.4 Etiquetas por proyecto y relación muchos a muchos 7

6.5 Valores permitidos del estado 7

6.6 Borrado lógico mediante el atributo habilitado 7

6.7 Seguridad y validación de datos 7

6.8 Por qué no se usó herencia 8

7. Arquitectura en capas 9

7.1 Responsabilidad de cada capa 10

8. Correspondencia entre el diagrama y el código 10

9. Reglas garantizadas también en la base de datos 11

10. Conclusión 12

Referencias 12

1. Introducción

El presente informe documenta el diseño orientado a objetos del sistema **Trello-POO**, desarrollado para la asignatura de Programación Orientada a Objetos a partir del contexto "Gestión de Proyectos tipo Trello". El objetivo de esta etapa es comprender el dominio del problema y transformarlo en un modelo de clases que luego se implementa en Python.

El documento presenta las clases identificadas y su responsabilidad, el diagrama de clases UML, las relaciones y multiplicidades entre clases, y las principales decisiones de diseño con su justificación. Además, describe la arquitectura en capas utilizada en la implementación y muestra cómo cada operación del diagrama se refleja en el código de la aplicación.

El diagrama se elaboró en draw.io y se entrega también como archivo editable (Diagrama_Clases_Trello_v3.drawio).

2. Contexto y alcance

Una organización necesita gestionar proyectos mediante tableros visuales. Cada proyecto puede contar con un tablero donde el trabajo se organiza en **listas** y **tarjetas**. Las tarjetas representan tareas con título, descripción, fecha límite y estado; pueden moverse entre listas, asignarse a un responsable y llevar etiquetas y comentarios. Un proyecto tiene varios participantes y un usuario puede participar en varios proyectos. El sistema debe permitir consultar el avance del trabajo.

A partir de ese enunciado se identificaron los sustantivos que tienen identidad propia, atributos y comportamiento (candidatos a clase), las relaciones entre ellos y las reglas que el sistema debe garantizar. El modelo resultante se implementó en Python con arquitectura en capas.

Alcance: gestión de usuarios, proyectos y participantes, tablero con listas, tarjetas (crear, modificar, mover, asignar responsable, cambiar estado, eliminar) y reporte de avance. Las etiquetas y los comentarios están modelados y presentes en la base de datos con datos de prueba.

3. Identificación de clases

Cada clase responde a un elemento del enunciado. La tabla resume su responsabilidad y por qué se modeló como clase y no como un simple atributo.

Clase	Responsabilidad	Por qué es una clase
Usuario	Persona que usa el sistema: datos de identificación y acceso.	Tiene identidad propia (RUT, correo) y participa en muchos proyectos; no pertenece a uno solo.
Proyecto	Agrupa el trabajo, sus participantes, su tablero y sus etiquetas. Calcula el avance.	Es el contenedor principal del dominio y define el ciclo de vida del tablero.
UsuarioProyecto	Participación de un usuario en un proyecto, con su rol y fecha de ingreso.	La relación muchos a muchos tiene datos propios (rol, fecha): por eso es una clase de asociación.
Tablero	Espacio visual del proyecto que contiene las listas.	El enunciado lo nombra explícitamente y tiene nombre y fecha propios.
Lista	Columna del tablero (ej. "Por hacer", "En curso", "Hecho") con un orden.	Tiene posición y contiene tarjetas; mover una tarjeta es cambiar su lista.
Tarjeta	Tarea concreta: título, descripción, fechas, estado y responsable.	Es el objeto con más comportamiento: se mueve, se asigna, cambia de estado y vence.
Etiqueta	Marca de clasificación (nombre y color) definida por proyecto.	Se reutiliza en muchas tarjetas del mismo proyecto.
TarjetaEtiqueta	Vincula tarjetas con etiquetas.	Resuelve la relación muchos a muchos entre Tarjeta y Etiqueta (clave primaria compuesta).
Comentario	Texto que un usuario deja en una tarjeta.	Tiene autor, fecha y texto propios; una tarjeta puede tener muchos.

4. Diagrama de clases

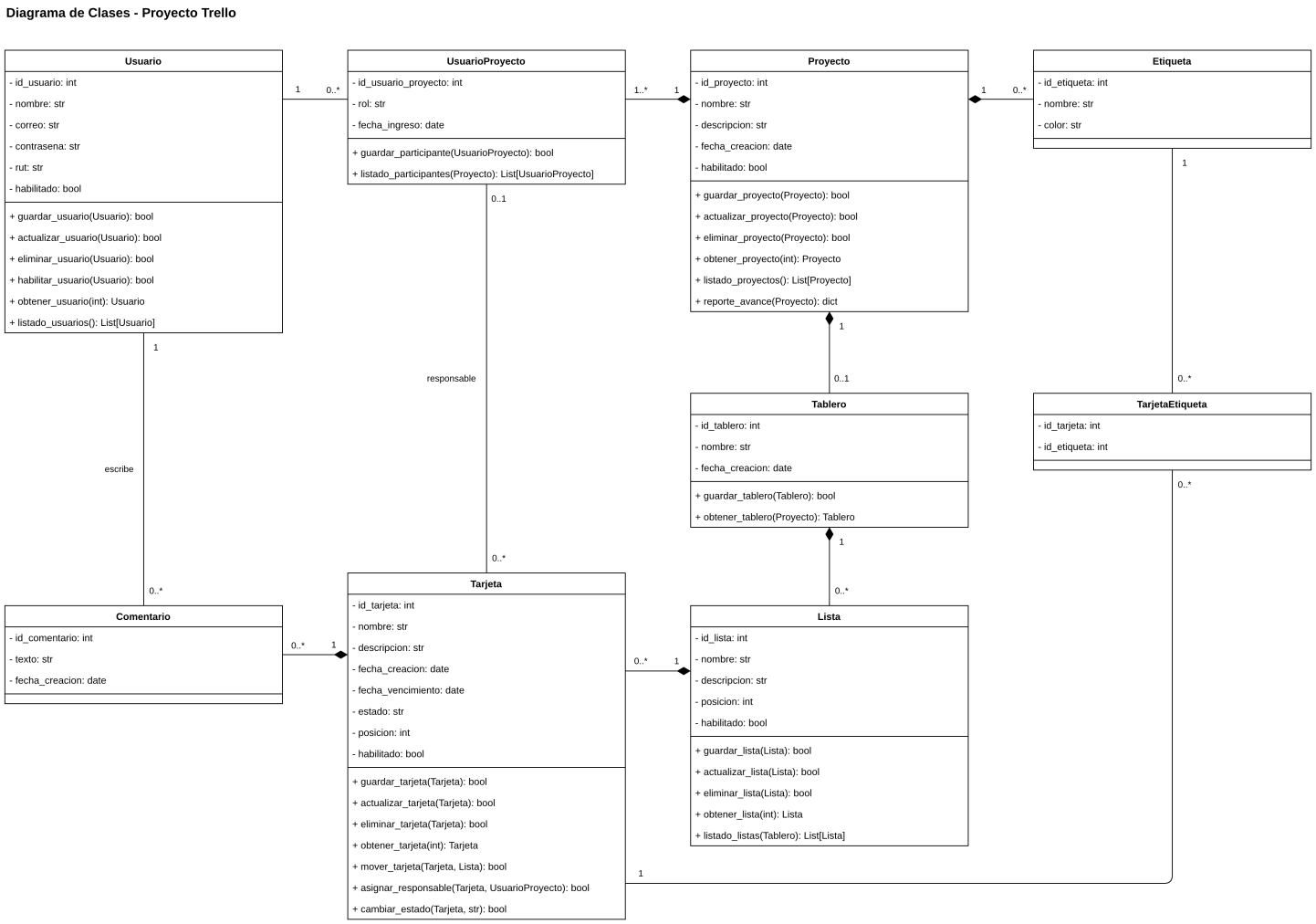


Figura 1. Diagrama de clases UML. Archivo editable: Diagrama_Clases_Trello_v3.drawio.

5. Relaciones y multiplicidades

Se usaron dos tipos de relación. La **composición** (rombo negro) se usó cuando la parte no tiene sentido sin el todo: si se elimina el todo, la parte deja de existir. La **asociación** se usó cuando ambos objetos existen de forma independiente.

Relación	Tipo	Multiplicidad	Justificación
Usuario – UsuarioProyecto	Asociación	1 — 0..*	Un usuario puede participar en cero o más proyectos; cada participación es de un solo usuario.
Proyecto – UsuarioProyecto	Composición	1 — 1..*	Un proyecto tiene al menos un participante: quien lo crea queda como administrador. Sin proyecto, la participación no existe.
Proyecto – Tablero	Composición	1 — 0..1	El enunciado dice "cada proyecto *puede* contar con un tablero": cero o uno. El tablero no existe fuera de su proyecto.
Tablero – Lista	Composición	1 — 0..*	Las listas pertenecen a un único tablero y no se comparten.
Lista – Tarjeta	Composición	1 — 0..*	Toda tarjeta está siempre en exactamente una lista; moverla cambia su lista, no la duplica.
UsuarioProyecto – Tarjeta	Asociación	0..1 — 0..*	Una tarjeta puede no tener responsable todavía; un participante puede ser responsable de muchas.
Proyecto – Etiqueta	Composición	1 — 0..*	Cada proyecto define sus etiquetas; el nombre es único dentro del proyecto.
Tarjeta – TarjetaEtiqueta – Etiqueta	Asociación (N:M)	1 — 0..* / 1 — 0..*	Una tarjeta puede tener varias etiquetas y una etiqueta se usa en varias tarjetas.
Tarjeta – Comentario	Composición	1 — 0..*	Un comentario siempre pertenece a una tarjeta y no tiene sentido sin ella.
Usuario – Comentario	Asociación	1 — 0..*	Cada comentario tiene un autor; el usuario existe aunque no comente.

6. Decisiones de diseño principales

6.1 UsuarioProyecto como clase de asociación

La relación entre usuarios y proyectos es muchos a muchos, pero además cada participación tiene información propia: el **rol** (administrador o miembro) y la **fecha de ingreso**. Esos datos no pertenecen ni al usuario (que tiene roles distintos en cada proyecto) ni al proyecto. Por eso la relación se modeló como una clase, con una restricción de unicidad (usuario, proyecto) para que nadie participe dos veces en el mismo proyecto.

6.2 El responsable de una tarjeta es un participante, no un usuario cualquiera

La tarjeta se relaciona con **UsuarioProyecto** y no directamente con Usuario. Así, el propio modelo garantiza que solo pueda ser responsable alguien que participa en el proyecto de esa tarjeta. Si la relación fuera con Usuario, se podría asignar una tarea a una persona ajena al proyecto y habría que controlarlo solo con código.

6.3 Cadena de composición Proyecto → Tablero → Lista → Tarjeta

El enunciado describe una jerarquía clara: el proyecto contiene un tablero, el tablero contiene listas y las listas contienen tarjetas. Ninguna de estas partes existe por sí sola, por lo que se usó composición. La multiplicidad **0..1** del tablero se refuerza en la base de datos con una restricción UNIQUE sobre el proyecto del tablero. Mover una tarjeta solo se permite entre listas del **mismo** tablero.

6.4 Etiquetas por proyecto y relación muchos a muchos

Las etiquetas pertenecen al proyecto para que cada equipo defina las suyas (por ejemplo, "Urgente" o "Diseño") sin mezclarse con otros proyectos. Como una tarjeta puede tener varias etiquetas y una etiqueta se aplica a varias tarjetas, se usó la clase TarjetaEtiqueta con clave primaria compuesta.

6.5 Valores permitidos del estado

El atributo estado de la tarjeta es un texto, pero solo puede ser *pendiente*, *en progreso* o *completada*. Estos valores se definen en un solo lugar de la aplicación (data_aplicacion.py), se validan en la capa de negocio y además con una restricción CHECK en la base de datos. Así el reporte de avance, que cuenta las tarjetas por estado, siempre trabaja con valores válidos.

6.6 Borrado lógico mediante el atributo habilitado

Usuario, Proyecto, Lista y Tarjeta tienen el atributo **habilitado**. Eliminar no borra la fila: la marca con 0. Un borrado físico rompería el historial (comentarios, participaciones, tareas asignadas) o sería rechazado por las claves foráneas. Además es reversible: un usuario deshabilitado puede volver a habilitarse. Los listados y el reporte solo consideran registros habilitados, y una lista con tarjetas no puede eliminarse.

6.7 Seguridad y validación de datos

La contraseña se guarda como **hash bcrypt**, nunca en texto plano. El RUT se valida con el algoritmo de dígito verificador (módulo 11) y se guarda con un formato único; RUT y correo no pueden repetirse. Las credenciales de la base de datos se leen desde un archivo .env que no se sube al repositorio.

6.8 Por qué no se usó herencia

Se evaluó la herencia, pero en este dominio no hay una relación "es un" real: una tarjeta no es un tipo de lista, ni un comentario un tipo de tarjeta. Forzar una jerarquía agregaría complejidad sin beneficio. La única herencia está en la implementación: todos los modelos heredan de **BaseModel** (Peewee), que comparte la conexión a la base de datos.

7. Arquitectura en capas

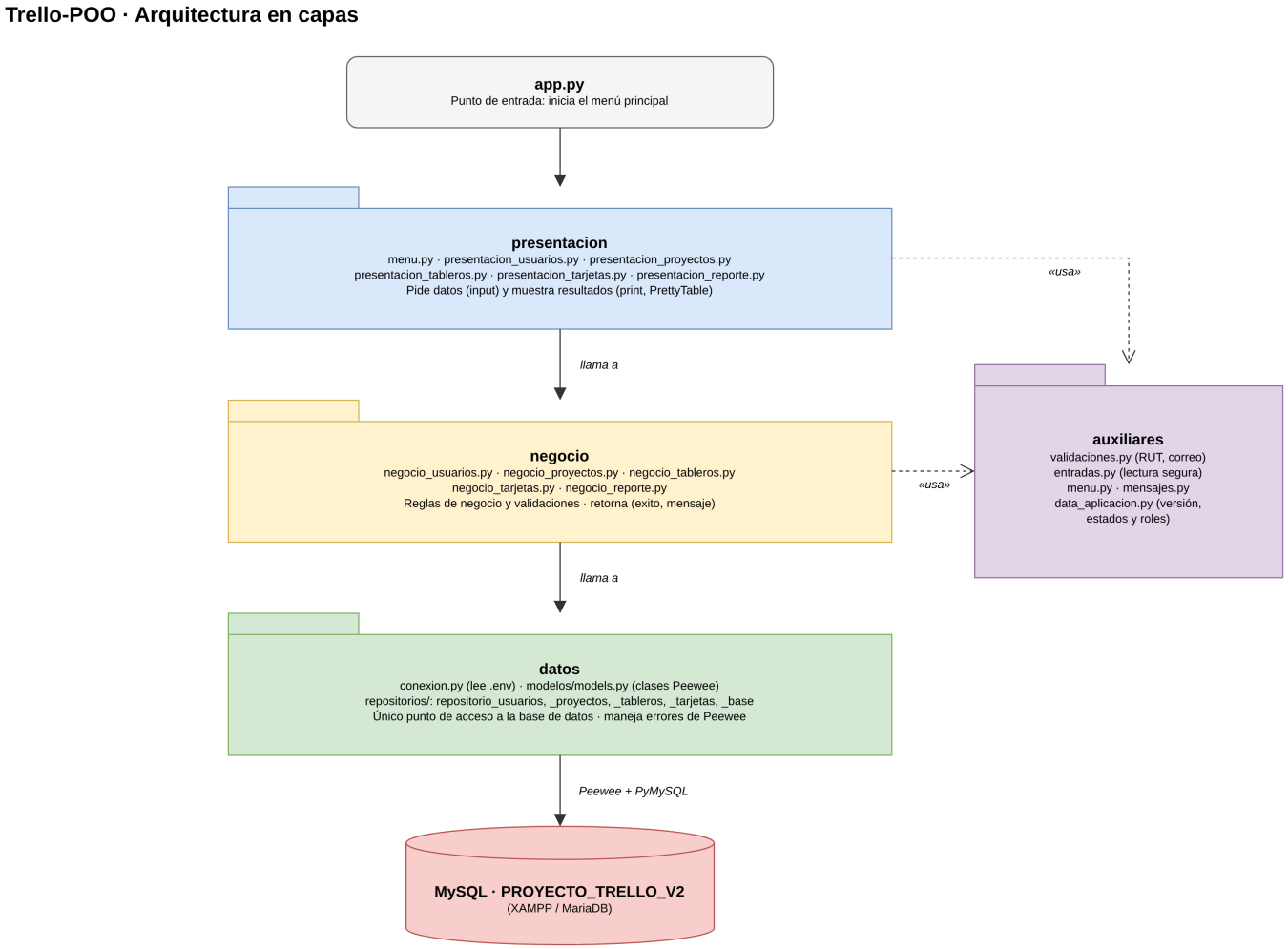


Figura 2. Arquitectura en capas de la aplicación.

7.1 Responsabilidad de cada capa

Capa	Responsabilidad	No hace
presentacion	Menús de consola: pide datos con input() y muestra resultados con PrettyTable.	No aplica reglas de negocio ni consulta la base de datos.
negocio	Reglas del sistema y validaciones. Cada operación retorna (éxito, mensaje).	No usa input() ni print() para pedir datos; no escribe SQL.
datos	Conexión (.env), modelos Peewee y repositorios: único acceso a MySQL y manejo de errores.	No decide reglas de negocio.
auxiliares	Herramientas compartidas: validación de RUT y correo, lectura segura, menús, mensajes.	No depende de ninguna otra capa.

Las dependencias van en una sola dirección (presentación → negocio → datos). Esto permite cambiar la interfaz (por ejemplo, a una interfaz gráfica) sin tocar las reglas, o cambiar el motor de base de datos sin tocar la presentación. Los errores inesperados se capturan en el menú y se muestran sin cerrar la aplicación.

8. Correspondencia entre el diagrama y el código

En el diagrama, las operaciones se dibujan dentro de la clase a la que conceptualmente pertenecen. En la implementación, siguiendo la arquitectura en capas, esas operaciones se programaron como funciones de la capa de negocio que reciben y modifican los objetos del modelo (clases Peewee en datos/modelos/models.py). La tabla muestra dónde está cada una.

Operación en el diagrama	Implementación (Python)
guardar_usuario / actualizar_usuario	negocio_usuarios.crear_usuario() / modificar_usuario()
eliminar_usuario / habilitar_usuario	negocio_usuarios.cambiar_habilitado()
obtener_usuario / listado_usuarios	negocio_usuarios.obtener_usuario() / lista_usuarios()
guardar_participante / listado_participantes	negocio_proyectos.agregar_participante() / lista_participantes()
guardar_proyecto / actualizar_proyecto / eliminar_proyecto	negocio_proyectos.crear_proyecto() / modificar_proyecto() / eliminar_proyecto()
obtener_proyecto / listado_proyectos	negocio_proyectos.obtener_proyecto() / lista_proyectos()
reporte_avance	negocio_reporte.reporte_avance()
guardar_tablero / obtener_tablero	negocio_tableros.crear_tablero() / obtener_tablero()
guardar_lista / actualizar_lista / eliminar_lista	negocio_tableros.crear_lista() / modificar_lista() / eliminar_lista()
obtener_lista / listado_listas	negocio_tableros.obtener_lista() / repositorio_tableros.listado_listas()
guardar_tarjeta / actualizar_tarjeta / eliminar_tarjeta	negocio_tarjetas.crear_tarjeta() / modificar_tarjeta() / eliminar_tarjeta()
obtener_tarjeta	negocio_tarjetas.obtener_tarjeta()

Operación en el diagrama	Implementación (Python)
mover_tarjeta	negocio_tarjetas.mover_tarjeta()
asignar_responsable	negocio_tarjetas.asignar_responsable()
cambiar_estado	negocio_tarjetas.cambiar_estado()

9. Reglas garantizadas también en la base de datos

Las reglas más importantes se validan en la capa de negocio (para dar mensajes claros al usuario) y, además, en la base de datos, para proteger los datos aunque se inserten directamente desde phpMyAdmin.

Regla	Restricción en MySQL
RUT y correo no se repiten	UNIQUE en USUARIOS.RUT y USUARIOS.CORREO
Un usuario participa una sola vez por proyecto	UNIQUE (ID_USUARIO, ID_PROYECTO) en USUARIO_PROYECTO
Un proyecto tiene un solo tablero	UNIQUE en TABLERO.ID_PROYECTO
Estados válidos de la tarjeta	CHECK (ESTADO IN ('pendiente', 'en progreso', 'completada'))
Etiqueta única dentro del proyecto	UNIQUE (ID_PROYECTO, NOMETIQ) en ETIQUETA
El responsable participa en el proyecto	Clave foránea TARJETA.ID_RESPONSABLE → USUARIO_PROYECTO
Borrado lógico	Columna HABILITADO TINYINT(1) DEFAULT 1 en USUARIOS, PROYECTO, LISTA y TARJETA

10. Conclusión

El modelo propuesto representa los elementos del contexto (usuarios, proyectos, tableros, listas, tarjetas, etiquetas y comentarios) y las reglas que el sistema debe cumplir. Las decisiones más relevantes fueron modelar la participación como una clase de asociación con rol, relacionar al responsable de una tarjeta con esa participación para asegurar que pertenezca al proyecto, y usar composición para la jerarquía Proyecto, Tablero, Lista y Tarjeta.

La implementación en Python con arquitectura en capas permitió separar la interfaz, las reglas de negocio y el acceso a datos. Gracias a esa separación se incorporaron nuevas funciones, como modificar y eliminar con borrado lógico, sin alterar la estructura del sistema. Las reglas principales se validan tanto en la capa de negocio como en la base de datos, lo que entrega mensajes claros al usuario y protege la integridad de la información.

Referencias

Bailey, E. (2026). *Contextos de proyectos para diseño de clases* [Material de la asignatura Programación Orientada a Objetos]. INACAP.

Bailey, E. (2026). *AP_N2_C2_POO: Proyecto de desarrollo modular con POO* [Repositorio de código]. GitHub. https://github.com/baileytorch/AP_N2_C2_POO

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3.ª ed.). Addison-Wesley.

Leifer, C. (s. f.). *Peewee documentation*. http://docs.peewee-orm.com

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), versión 2.5.1*. https://www.omg.org/spec/UML/2.5.1